

Tutorato calcolatori - 11/03/2026

1 Mockup 1

1. **3 Punti** Seleziona la rappresentazione binaria del numero 7825_{10}

- ☒ 0001 1110 1001 0001
☐ 0001 1110 1001 0000
☐ 0001 1110 1011 0001
☐ 0011 1110 1011 0001
☐ Nessuna delle risposte

Soluzione: Convertiamo direttamente il numero usando il metodo della divisione:

7825	1
3912	0
1956	0
978	0
489	1
244	0
122	0
61	1
30	0
15	1
7	1
3	1
1	1
0	

Dunque $7825_{10} = 0001\ 1110\ 1001\ 0001_2$

2. **3 Punti** Usando la rappresentazione binaria, effettua la sottrazione:
 $0001\ 0001\ 0000\ 1111\ 1010\ 1101\ 1100 - 0000\ 1000\ 0111\ 1110\ 1011\ 0000\ 0100$

- ☐ 0000 1000 1001 0000 1111 0001 1100
☐ 0000 1000 1001 0000 1111 0001 1000
☒ 0000 1000 1001 0000 1111 1101 1000
☐ 0000 1000 1001 0000 1111 1101 1100
☐ Nessuna delle risposte

Soluzione: Eseguiamo la soluzione:

	0001	0001	0000	1111	1010	1101	1100
–	0000	1000	0111	1110	1011	0000	0100
	0000	1000	1001	0000	1111	1101	1000

3. **3 Punti** Calcola la rappresentazione di -137_{10} in complemento a 2 su 8 bits

- ☐ 1000 1001
- ☐ 1111 0110
- ☐ 1111 1100
- ☐ 1011 0110
- ☒ Nessuna delle risposte

Soluzione: Il numero -137 in binario è:

137		1
68		0
34		0
17		1
8		0
4		0
2		0
1		1
0		

Il quale già usa **8 bits** il che significa che il numero **non è rappresentabile** in complemento a 2 con solamente **8 bits**.

4. 3 Punti Convertire il numero 36.875 in binario con rappresentazione a virgola fissa.
- ☐ 0010 0100.1010
 - ☒ 0010 0100.1110
 - ☐ 0010 0100.0110
 - ☐ 0010 0110.1110
 - ☐ Nessuna delle risposte

Soluzione: Separiamo la parte intera da quella frazionaria e convertiamole separatamente:

36		0		
18		0		
9		1		
4		0		
2		0		
1		1		
0				

.875		1
.75		1
.5		1
.0		

Quindi, $36.875_{10} = 0010\ 0100.1110_2$

5. 3 Punti Quale delle seguenti affermazioni è **FALSA**?
- ☐ Il linguaggio *Assembly* è strettamente legato alla CPU su cui il programma dovrà eseguire.
 - ☒ Una caratteristica dei programma scritti in linguaggio *Assembly* è la sua portabilità.
 - ☐ Per essere eseguito, un programma *Assembly* deve essere tradotto in linguaggio macchina da un compilatore

- ☐ Al livello più basso, la CPU può capire solo programmi scritti in linguaggio macchina
- ☐ Il linguaggio *Assembly* codifica le istruzioni macchina tramite codici mnemonici.

Soluzione: La affermazione FALSA è: **Una caratteristica dei programma scritti in linguaggio *Assembly* è la sua portabilità.**

Questo in quanto il linguaggio *Assembly* è strettamente legato alla CPU su cui il programma dovrà eseguire.

Il programma scritto in *Assembly* non è ancora eseguibile dalla macchina deve essere convertito in linguaggio macchina da un assemblatore.

Il più basso livello di astrazione la CPU comprende solamente linguaggio macchina.

In *Assembly* codifichiamo le istruzioni tramite dei codici mnemonici immutabili.

6. 3 Punti Usando la rappresentazione binaria, svolgere la sottrazione $0100\ 1001\ 1110\ 1001 - 0x3A18$
- ☐ 1111 1101 0101
 - ☐ 1100 1101 0101
 - ☐ 1100 1101 0001
 - ☒ 1111 1101 0001
 - ☐ Nessuna delle risposte

Soluzione: Prima di tutto convertiamo il numero esadecimale in binario:

3	A	1	8
0011	1010	0001	1000

Ora, eseguiamo la sottrazione:

	0100	1001	1110	1001
–	0011	1010	0001	1000
		1111	1101	0001

2 Mockup 2

1. 3 Punti Rappresenta il numero decimale -1450.78 nella codifica dello standard IEEE754
- ☒ 1100 0100 1011 0101 0101 1000 1111 0110
 - ☐ 1110 0100 1011 0101 0101 1000 1111 0110
 - ☐ 1100 0100 1011 0101 0101 1000 1111 1110
 - ☐ 1100 0100 1011 0111 0101 1000 1111 0110
 - ☐ Nessuna delle risposte

Soluzione: Il primo bit è 1 in quanto il segno è negativo.

1450.78_{10} convertito in binario è:

		.78	1
		.56	1
1450	0	.12	0
725	1	.24	0
362	0	.48	0
181	1	.96	1
90	0	.92	1
45	1	.84	1
22	0	.68	1
11	1	.36	0
5	1	.72	1
2	0	.44	0
1	1	.88	1 (Ultimo Bit)
0		.76	1
		...	...

Come si può notare la parte frazionaria è periodica, quindi prendiamo in considerazione solo i primi 23 bit escluso il 1:

101 1010 1010.1100 0111 1010 1

, inoltre aggiungiamo un 1 in quanto la ricorsione non termina.

$$\begin{array}{r}
 \dots \quad 1111 \quad 0101 \\
 + \quad 0000 \quad 0001 \\
 \hline
 \dots \quad 1111 \quad 0110
 \end{array}$$

Ora normalizziamo il numero in modo da avere un solo bit prima della virgola, ovvero:

1.011 0101 0101 1011 0001 1101 0110 $\cdot 2^{10}$

L'esponente è 10 trasformato con l'offset di 127 è $137_{10} = 1000 \ 1001_2$

Quindi, la rappresentazione in IEEE754 di -1450.78 è:

$$\underbrace{1}_{\text{segno}} \underbrace{1000 \ 1001}_{\text{esponente}} \underbrace{0110 \ 1010 \ 1011 \ 0001 \ 1110 \ 110}_{\text{mantissa}}$$

= 1100 0100 1011 0101 0101 1000 1111 0110

2. 3 Punti Usando la rappresentazione binaria, effettua la somma $5678_{10} + 0001 \ 0010 \ 0101 \ 1010_2$

- ☐ 0011 0001 0001 0001
- ☐ 0010 0001 0001 0001
- ☒ 0010 1000 1000 1000
- ☐ 0010 0001 0001 0011
- ☐ Nessuna delle risposte

Soluzione: Prima di tutto, convertiamo 5678_{10} in binario:

5678	0
2839	1
1419	1
709	1
354	0
177	1
88	0
44	0
22	0
11	1
5	1
2	0
1	1
0	

Quindi, $5678_{10} = 0001\ 0110\ 0010\ 1110_2$

Ora, sommiamo i due numeri binari:

$$\begin{array}{rcccc}
 & 0001 & 0110 & 0010 & 1110 \\
 + & 0001 & 0010 & 0101 & 1010 \\
 \hline
 & 0010 & 1000 & 1000 & 1000
 \end{array}$$

3. **3 Punti** Codifica il numero esadecimale $0x1087D0F187$ secondo lo standard IEEE754?

- ☐ 0101 0001 1000 0100 0011 1110 1100 1100
☐ 0101 0011 1000 0100 0011 1110 1000 1000
☐ 0101 0001 1000 0100 0011 1110 1000 1100
☒ 0101 0001 1000 0100 0011 1110 1000 1000
☐ Nessuna delle risposte

Soluzione: Per prima cosa convertiamo il numero esadecimale in binario:

1	0	8	7	D	0	F	1	8	7
0001	0000	1000	0111	1101	0000	1111	0001	1000	0111

Ora prendiamo i primi 23 bits senza il primo 1 sotto inteso come mantissa

1. $0000\ 1000\ 0111\ 1101\ 0000\ 111 \cdot 2^{36}$ l'esponente calcolato è $36 + 127 = 163$ che convertito in binario è: $163_{10} = 1010\ 0011$. Il primo bit è 0 in quanto il numero è positivo, e dunque il numero codificato è:

$$\underbrace{0}_{\text{segno}} \underbrace{1010\ 0011}_{\text{esponente}} \underbrace{0000\ 1000\ 0111\ 1101\ 0000\ 111}_{\text{mantissa}}$$

$$= 0101\ 0001\ 1000\ 0100\ 0011\ 1110\ 1000\ 0111$$

Ora in quanto abbiamo selezionato solamente una parte del numero originale bisogna aggiungere 1 alla mantissa portando alla rappresentazione finale:

$$\underbrace{0}_{\text{segno}} \underbrace{1010\ 0011}_{\text{esponente}} \underbrace{0000\ 1000\ 0111\ 1101\ 0001\ 000}_{\text{mantissa}}$$

$$= 0101\ 0001\ 1000\ 0100\ 0011\ 1110\ 1000\ 1000$$

4. **3 Punti** In un'architettura **RISC**, cosa succede se è necessario accedere ai dati in memoria?
- ☐ Si utilizzano istruzioni aritmetiche complesse che includono l'indirizzo di memoria.
 - ☒ **Si devono utilizzare esclusivamente le istruzioni dedicate *load* e *store*.**
 - ☐ Non è possibile accedere alla memoria.
 - ☐ La CPU sposta automaticamente i dati nei registri durante la decodifica.

Soluzione: La prima risposta è errata in quanto in una architettura **RISC** le istruzioni sono semplici e l'indirizzo non è incluso nelle istruzioni aritmetiche.

La terza risposta è errata in quanto è possibile accedere alla memoria

La quarta risposta è errata in quanto la CPU non sposta automaticamente i dati nei registri

La seconda risposta è corretta in quanto in una architettura **RISC** è necessario utilizzare esclusivamente le istruzioni dedicate *load* e *store* per accedere alla memoria.

5. **3 Punti** Qual è la rappresentazione in complemento a 1 (su 4 bit) del numero decimale -6 ?
- ☐ 0110
 - ☒ 1001
 - ☐ 1010
 - ☐ 1011
 - ☐ Nessuna delle risposte

Soluzione: Il numero 6 in binario su 4 bit è:

$$\begin{array}{r|l} 6 & 0 \\ 3 & 1 \\ 1 & 1 \\ 0 & \end{array}$$

Quindi, $6_{10} = 0110_2$, il suo complemento a 1 si ottiene invertendo i bit, ovvero 1001_2 . Quindi, la rappresentazione in complemento a 1 di -6 è 1001_2 .

6. **3 Punti** Qual è la rappresentazione in base 2 del numero 25_{13} ?
- ☐ 0011 0111
 - ☒ 0001 1111
 - ☐ 0011 1011
 - ☐ 0010 1110
 - ☐ Nessuna delle risposte

Soluzione: Per convertire un numero dalla base 13 alla base 2, prima convertiamo il numero in base 10 e poi in base 2 usando la regola generale per la conversione tra basi.

$$\begin{aligned} n_a &= \sum_{i=0}^k d_i \cdot a^i \\ 25_{13} &= 2 \cdot 13^1 + 5 \cdot 13^0 \\ &= 26 + 5 = 31_{10} \end{aligned}$$

Ora convertiamo 31_{10} in base 2:

31	1
15	1
7	1
3	1
1	1
0	

Quindi $25_{13} = 0001\ 1111_2$.